CampusClaw 校园知识库

# 基于 Token 的身份认证与知识库问答实现方案

—— 结合现有代码的实现梳理 ——

**课程：**服务端开发实践（0930 课后任务）

**技术栈：**Flask + MySQL 8.0 + Qdrant + OpenAI 兼容模型网关 + Nginx（Docker Compose 四服务）

**日期：**2026 年 9 月 30 日

# 目录

**一、系统概述**

1.1 系统定位与总体架构　1.2 代码分层与关键文件

**二、基于 Token 的身份认证实现方案**

2.1 方案选型：JWT + 服务端会话登记的混合方案　2.2 数据结构　2.3 Token 结构
2.4 登录与签发　2.5 请求鉴权　2.6 登出与撤销　2.7 前端配合　2.8 安全要点

**三、基于知识库的问答实现方案**

3.1 总体链路　3.2 知识入库：上传事务与索引构建　3.3 文本切片策略
3.4 三种检索模式　3.5 /ask 有依据问答（RAG）　3.6 模型网关适配
3.7 数据存储分工与班级隔离　3.8 降级与容错

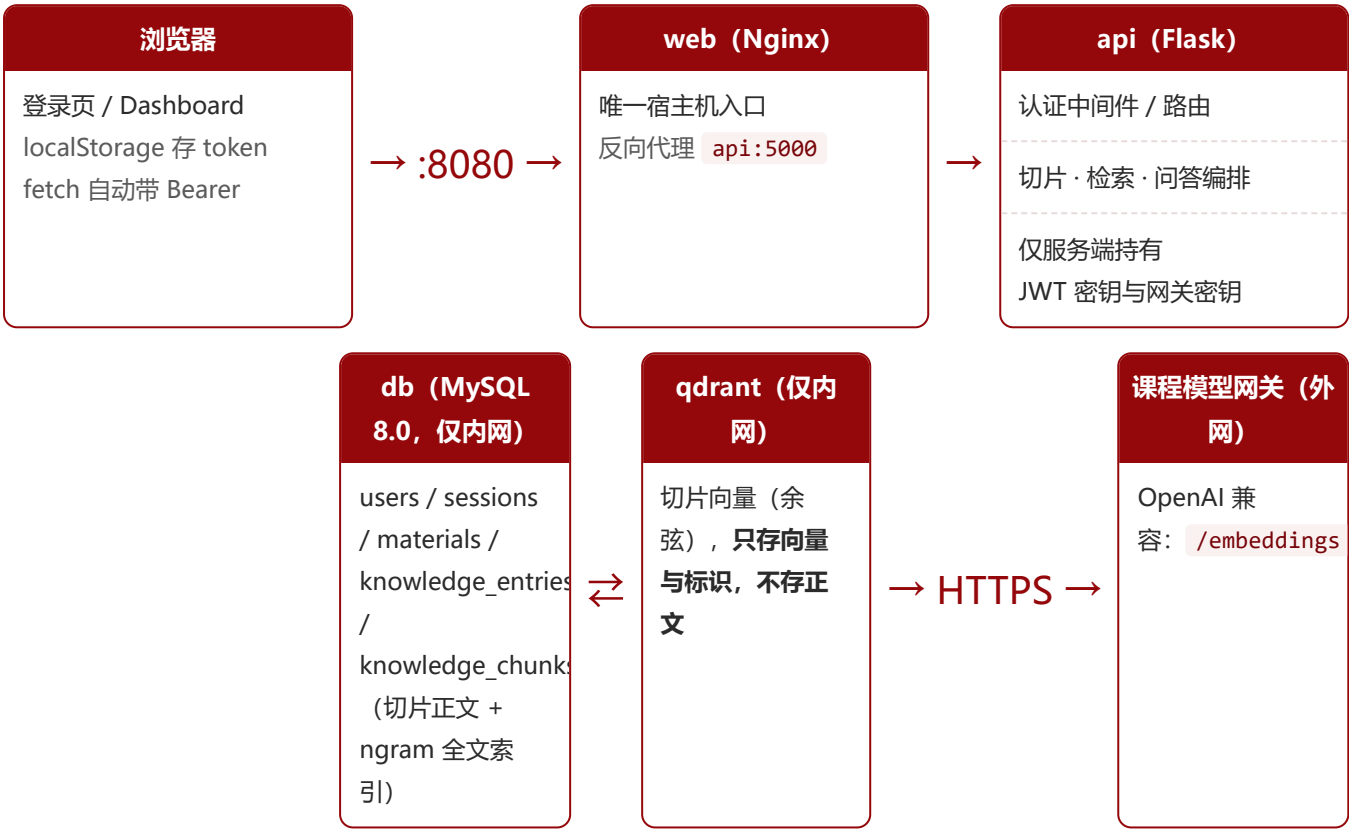
**四、接口清单**

**五、验证情况**

# 一、系统概述

## 1.1 系统定位与总体架构

CampusClaw 是面向班级教学场景的最小可用知识库系统，提供三条核心能力闭环：**账号密码登录与角色/班级鉴权、班级材料上传与管理、限定班级范围、可溯源的检索与"有依据才回答"的问答。**系统以 Docker Compose 四服务为唯一交付形态，仅 web 服务映射宿主机端口：




说明：浏览器与模型网关之间不存在直连；嵌入与对话密钥只存在于 api 的服务端环境变量（`.env`）中。api 启动时自带数据库连接重试（30 次 × 2 秒，仅重试 errno 2003/2013 类连接级错误）。

## 1.2 代码分层与关键文件

| 层 | 文件 | 职责 |
| --- | --- | --- |
| 路由层 routes | `auth.py` | 登录签发 / 登出撤销 / 当前用户 |
| | `search.py` 、 `ask.py` | 检索接口、问答接口 |
| | `materials.py` 、 `pages.py` | 上传/下载/重建索引、登录页与 Dashboard 页面 |
| 中间件 middleware | `auth.py` | `@auth_required` ：Bearer 解析与会话校验 |
| | `role.py` | `@role_required` ：教师/学生角色授权（403） |
| 服务层 services | `token_auth.py` | JWT 签发与验签（HS256） |
| | `chunking.py` / `indexing.py` | 文本切片；落库→嵌入→写向量库的索引编排 |
| | `retrieval.py` | 关键字/向量/混合（RRF）三路检索与结果装配 |
| | `answer.py` | 有依据问答：检索、拼提示词、调对话模型、出引用 |
| | `model_gateway.py` / `vector_store.py` | OpenAI 兼容网关客户端；Qdrant 适配 |
| 仓储层 repositories | `users / sessions / materials / knowledge / chunks` | 全部 SQL；所有读查询强制带班级条件 |

# 二、基于 Token 的身份认证实现方案

## 2.1 方案选型：JWT + 服务端会话登记（混合方案）

认证载体采用标准 **JWT Bearer Token**：登录成功后服务端签发 token，浏览器后续请求一律携带 `Authorization: Bearer <token>`。但系统没有采用"纯无状态 JWT"，而是把每个 token 的唯一标识 `jti` 登记进服务端 `sessions` 表，原因是纯无状态方案有两个无法接受的问题：

- **无法主动撤销**：纯 JWT 在过期前始终有效，用户点"退出登录"也无法让它立即失效；
- **身份快照陈旧**：若把 role/class_id 放进 payload，改库后已签发 token 仍携带旧身份。

因此本方案的分工是：**JWT 只做防篡改的"身份信封"**（证明 userId 没被伪造、是否在有效期）；**是否仍被允许访问以 sessions 表为准，角色与班级每请求实时读 users 表。**由此同时得到"token 方案"与"登出立即失效、身份实时生效"两条特性。

## 2.2 数据结构

认证相关两张表（`migrations/001、002`）：

```
users(id, username UNIQUE, password_hash TEXT,   -- bcrypt 哈希
      role ENUM('teacher','student'), class_id)

sessions(id VARCHAR(128) PRIMARY KEY,            -- 存 JWT 的 jti（uuid4().hex，32 字符）
         user_id BIGINT, expires_at DATETIME,    -- 与 token 的 exp 对齐
         FK user_id ON DELETE CASCADE)
```

sessions 表是此前服务端会话期就存在的表，主键改为承载 jti 后**无需新增迁移**；`expires_at` 上建索引便于后续清理过期会话。

## 2.3 Token 结构

签发逻辑见 `app/services/token_auth.py` 的 `issue()`：算法 **HS256**（对称签名，密钥 `JWT_SECRET` 只在服务端），payload 刻意只放四个声明：

```
{
  "sub": "1",                 // 用户 id（字符串形式）
  "jti": "a1b2…（uuid4 hex）", // 会话/撤销标识，同时写入 sessions.id
  "iat": 1727…,               // 签发时间
  "exp": 1727…                // 过期时间 = iat + SESSION_TTL_HOURS
}
```

> **关键决策：**payload **不含 role、class_id、密码等任何信息**。验签时通过 `options={"require": ["exp","sub","jti"]}` 强制三声明齐全；任何伪造、篡改签名、过期、格式错误都抛 `TokenInvalid`。

## 2.4 登录与签发流程（POST /auth/login）

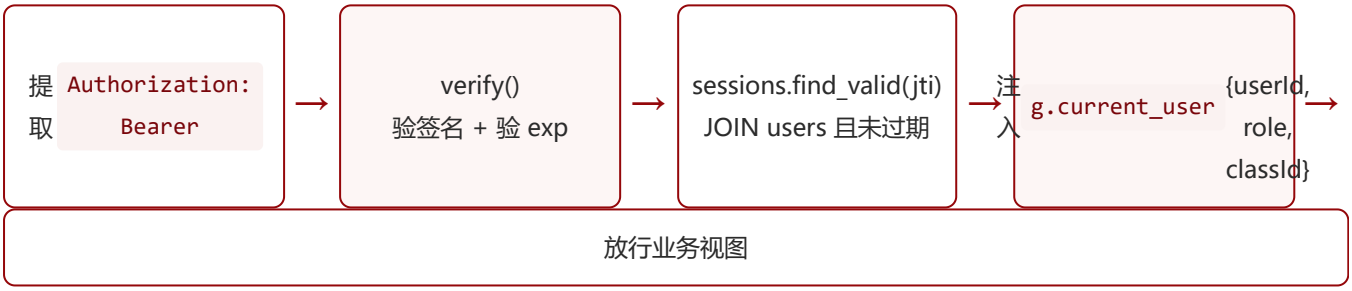
实现位于 `app/routes/auth.py`，全程不设置任何 Cookie：

1. 解析 JSON 取 username/password；任一为空直接返回 `401 {"error":"invalid credentials"}`。
2. `UserRepository.find_by_username()` 查 users 表；**无论用户是否存在都执行一次 bcrypt 校验**——用户不存在时使用预计算的 `_DUMMY_HASH` 走同样耗时路径，防止通过响应耗时枚举用户名。
3. 用户不存在或 `bcrypt.checkpw()` 失败，统一返回相同的 `401 invalid credentials`。
4. 校验通过：`token_auth.issue(config, user_id)` 生成 `(token, jti, expires_at)`。
5. `sessions_repo.create()` 把 jti 登记入库（与 token 内容一一对应）。
6. 返回 `200 {"token": "<JWT>", "user": {"id","role","classId"}}`，**响应无 Set-Cookie**。

重复登录会得到不同 jti 的新 token，旧 token 不受影响（支持多端同时在线）；`JWT_SECRET` 与 `SESSION_TTL_HOURS` 均为必填环境变量，缺失时应用拒绝启动。

## 2.5 请求鉴权流程（@auth_required）

除 `/health` 与页面外壳外，所有业务接口都挂 `@auth_required`（`app/middleware/auth.py`）：




其中 `find_valid()` 的 SQL 为 `sessions JOIN users … WHERE s.id=%s AND s.expires_at > NOW()`：一条查询同时完成"会话是否仍有效"与"取实时身份"。任一环节失败（无头、头格式错、签名无效、过期、jti 不在表中、已过期）一律拒绝：

- API 请求（Accept 含 application/json 或非 html）→ `401 {"error":"unauthorized"}`；
- 浏览器页面请求（Accept: text/html）→ `302` 跳转 `/login?next=<原路径>`。

角色授权由叠加在其后的 `@role_required("teacher")` 完成，角色不匹配返回 `403`（如学生上传、重建索引）。

## 2.6 登出与撤销（POST /auth/logout）

登出接口本身也要先通过 `@auth_required`，然后按当前请求的 `g.token_jti` **删除 sessions 表对应行**，返回 `200 {"ok":true}`。删除后：JWT 本身仍未过期，但中间件 `find_valid(jti)` 查不到记录，该 token **立即失效**（测试覆盖"同一 token 登出后再访问 /auth/me 返回 401"）。

## 2.7 前端配合（pages.py 内联 JS）

- 登录成功后把 token 存入 `localStorage`（key：`cc_token`），跳转到 `next` 或 /dashboard；
- 页面内所有接口调用统一走 `api(path, opts)` 封装：自动注入 `Authorization: Bearer` 头；一旦收到 401 立即清除本地 token 并跳回登录页，避免散落地手写 fetch 漏带头；
- 文件下载/打开不能用浏览器地址栏直链（导航请求无法附加 Authorization 头），统一 `fetch → Blob → a[download]` / `window.open(blobURL)` 实现。

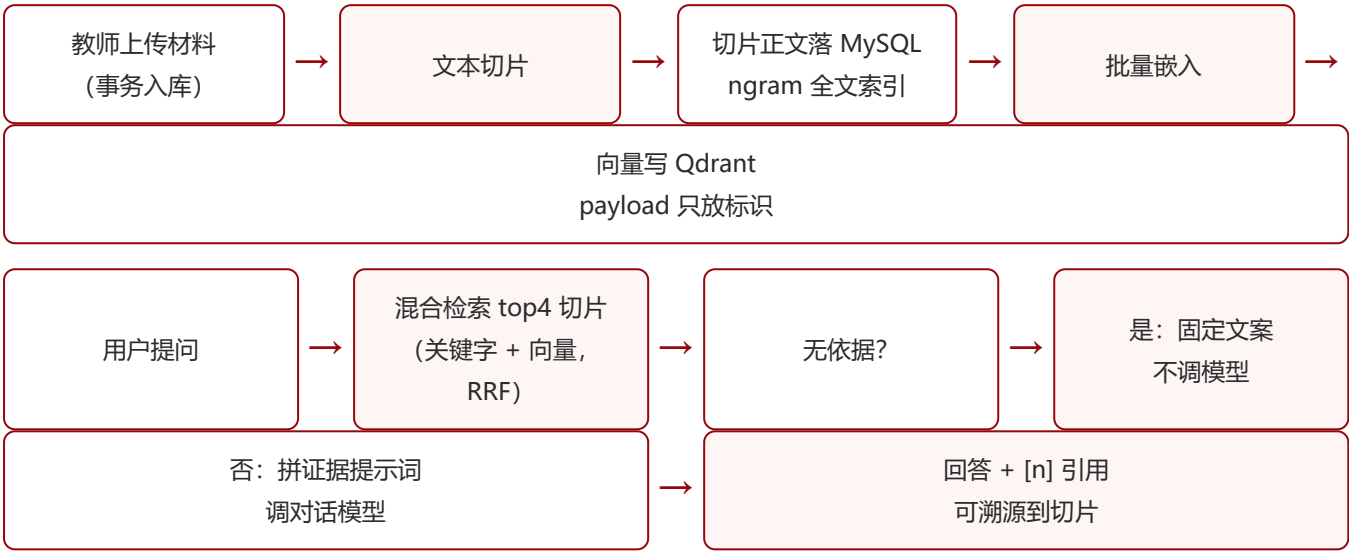
## 2.8 安全要点小结

| 要点 | 实现方式（代码位置） |
| --- | --- |
| 密码不明文存储/比对 | users 表存 bcrypt 哈希；`bcrypt.checkpw` 校验（auth.py） |
| 防用户名枚举 | 用户不存在也执行一次 bcrypt（`_DUMMY_HASH` 恒时路径），错误文案完全一致 |
| 防伪造/防篡改/防过期 | HS256 签名 + 强制 exp/sub/jti 声明（token_auth.py） |
| 可即时撤销 | jti 写 sessions 表，登出删行；每请求查表（sessions.py / auth.py） |
| 身份实时生效 | role/class_id 不进 token，每请求 JOIN users 实时读取 |
| 密钥只在服务端 | `JWT_SECRET` 必填环境变量；前端只能看到自己的 token |
| 鉴权统一兜底 | 业务接口逐个挂 `@auth_required`；页面/API 区分 302/401 |
| 不使用 Cookie 会话 | 无 Set-Cookie，天然不存在基于 Cookie 的 CSRF 问题 |

# 三、基于知识库的问答实现方案

问答本质是检索增强生成（RAG）：**先在本班材料切片中检索证据，再把证据交给对话模型生成带引用的回答；检索不到证据就不调用模型。**整体分为"知识入库（离线侧）"与"检索问答（在线侧）"两段。

## 3.1 总体链路




## 3.2 知识入库：上传事务 + 提交后建索引

### （1）上传：先写文件，再单事务写双表（`services/upload.py`）

1. 教师角色校验在任何写操作之前（学生 403）。
2. 扩展白名单：仅 `.txt/.md`，否则 415；大小上限 `UPLOAD_MAX_SIZE_MB`（默认 50MB），否则 413。
3. 服务端生成 `uuid` 存储名，落盘到 `uploads/materials/<class_id>/`，**不信任客户端文件名。**
4. **一个事务**同时写 materials 与 knowledge_entries（正文全文），两表 class_id 一致；事务失败则回滚并删除已写文件，杜绝孤儿文件/孤儿记录。

### （2）索引：事务提交之后执行，失败不影响材料（`services/indexing.py`）

材料行提交成功后才调用 `index_material()`，即使索引全部失败，上传接口仍返回 201——这保证"先把资料存下来，索引可事后重建"。

## 3.3 文本切片策略（`services/chunking.py`，纯函数）

| 策略 | 规则 |
| --- | --- |
| `auto`（默认） | 约 800 字窗口、80 字重叠；切点优先选段落/换行/句号等自然边界（窗口后半段找边界，避免切出碎片） |
| `custom` | 窗口 100–2000 字、重叠率 0–50%（越界抛 ChunkParamError → 400）；可选剥离 URL/邮箱、折叠空白 |
| `hierarchy` | 按 `#` / `##` / `###` 标题切分（标题随所属段落），超长章节再按 auto 规则补切 |

每个切片记录 `chunk_index` 与在正文中的 `char_start/char_end`，使检索结果可以精确溯源到字符区间。

### （3）双存储写入与双状态分离

1. 切片先批量插入 MySQL `knowledge_chunks`，`index_status='ready'`、`embedding_status='pending'`，提交后**关键字检索立即可用**。
2. 调用嵌入网关 `/embeddings` 批量向量化，校验返回条数与切片数一致。
3. `ensure_collection()` 确认 Qdrant 集合存在（余弦距离），维度冲突显式报错。
4. upsert 向量点：**point.id = 切片主键 id**；payload 仅含 `chunk_id/class_id/material_id/knowledge_entry_id/chunk_index`，**绝不放正文**。成功后 embedding_status 置 ready。
5. 网关或 Qdrant 失败：捕获后把 embedding_status 置 `failed` 且不写半成品向量；正文切片仍 ready，关键字检索不受影响，教师可调 `/materials/<id>/reindex` 重建。

重建索引先按 material_id 尽力删除旧向量、删除旧切片，再重新走完整索引；启动时对"还没有切片的材料"做一次尽力 backfill，且绝不阻塞启动。

## 3.4 三种检索模式（`services/retrieval.py` + `repositories/chunks.py`）

| 模式 | 实现 |
| --- | --- |
| keyword | 只查 MySQL：`MATCH(chunk_text) AGAINST (%s IN NATURAL LANGUAGE MODE)`，基于 **ngram 解析器（token=2）** 的全文索引，天然适合中文短词；条件含 `class_id` 与 `index_status='ready'`。不依赖向量库与网关。 |
| vector | 先把 query 调嵌入网转向量，再在 Qdrant 做**带 class_id 过滤**的余弦检索，相似度阈值 `0.35`；返回的 id **再回 MySQL 按班级与 ready 状态二次复核**。 |
| hybrid（默认） | 关键字与向量两路独立检索后，用 **RRF（Reciprocal Rank Fusion，k=60）** 融合排序：score = Σ 1/(60 + 排名)；任一路缺席不影响另一路。 |

调参常量（`config.py`，可用环境变量覆盖）：`RETRIEVAL_VECTOR_THRESHOLD=0.35`、`RRF_K=60`、`SEARCH_PATH_LIMIT=20`（每路候选）、`SEARCH_FINAL_LIMIT=10`（最终条数）、`EXCERPT_LIMIT=200`（检索结果展示摘录长度）。最终结果由 `_assemble_with_limit()` 严格按融合排序从 MySQL 取正文与材料标题组装，每条命中携带：材料标题、切片序号、字符区间、正文摘录、材料链接——全部可溯源。

> **跨班与异常语义：**检索班级**只取自登录会话**，请求体里的 `class_id` 从不读取；跨班查询表现为 `200 + 空命中 + 固定提示`（而非 403/404，避免信息泄漏）。空 query → 400；向量/混合模式遇到网关或 Qdrant 故障 → **503 显式报错，不静默降级为关键字结果**，防止用户把残缺结果误当混合检索。

## 3.5 /ask 有依据问答（`services/answer.py`）

`POST /ask` 同样以 `g.current_user["classId"]` 为唯一班级来源，步骤：

1. 问题为空/纯空白 → 400。
2. 以 hybrid 模式检索，取 `ASK_TOP_K=4` 条切片，**摘录上限放宽到 2000 字**（问答提示词需要完整证据，若复用检索接口 200 字展示摘录会因证据被截断导致模型拒答）。
3. 先据命中构造 citations（序号、材料 id/标题、切片号、材料链接）。
4. **0 命中：直接返回固定文案"资料中未找到相关内容"，citations 为空，且完全不调用对话模型**（省成本、防幻觉）。
5. 有命中：组装消息——服务端 system 提示词（只能依据给定切片、每条结论用 [序号] 标注、资料不足就回固定文案、简体中文简短回答）+ 清洗后的历史 + 含全部切片正文的 user 消息——调对话网关（非流式）。
6. 返回 `{"answer", "citations"}`，回答中的 [1][2] 与 citations 顺序一致，前端据此渲染可点击的溯源链接。

> **提示词注入防护（输入侧）：**历史消息只保留 `user/assistant` 纯文本角色且**丢弃客户端传入的 system 角色**，并只取最近若干轮（最多 6 条）；向量与网关原始响应不回传浏览器。

## 3.6 模型网关适配（`services/model_gateway.py`）

- 面向课程提供的 **OpenAI 兼容网关**：嵌入 `POST {EMBED_BASE_URL}/embeddings`，对话 `POST {CHAT_BASE_URL}/chat/completions`（`stream:false`），均以服务端密钥 Bearer 调用；
- 嵌入结果按 `index` 排序，保证向量与切片顺序一一对应；对话取 `choices[0].message.content`；
- 错误分级：未配置/不可达/超时/5xx/响应不可解析 → `GatewayUnavailable`（接口映射为 503）；网关 4xx 拒绝 → `GatewayRejected`；**不回显网关响应体、不记录密钥**；
- 未配置密钥时系统照常启动：关键字检索、上传下载、认证授权全部可用，向量/混合检索与问答返回 503 并提示依赖不可用。

## 3.7 数据存储分工与班级隔离

| 存储 | 内容 |
| --- | --- |
| MySQL `knowledge_entries` | 材料正文全文与来源（每材料一行） |
| MySQL `knowledge_chunks` | **切片正文的唯一存放地**；char 区间、策略、双状态；ngram FULLTEXT；(material_id, chunk_index) 唯一 |
| Qdrant point | id=切片 id + 2048 维向量 + 仅标识 payload（含 class_id 便于向量侧过滤） |

班级隔离由三道闸保证：① 路由层班级只取自服务端会话，忽略请求体 class_id；② 向量检索在 Qdrant 侧带 class_id 过滤；③ 所有 id 回到 MySQL 取正文时再带 `class_id` 与 ready 条件复核（`get_ready_by_ids_scoped`），任何外班 id 直接缺席。

## 3.8 降级与容错一览

| 故障 | 系统行为 |
| --- | --- |
| MySQL 未就绪 | api 启动重试 30×2s；超时则退出由容器重启策略接管 |
| 嵌入/向量故障（上传时） | 材料与切片正常保留，embedding_status=failed，关键字检索可用，可重建索引 |
| 网关未配置/不可达（检索问答时） | keyword 正常；vector/hybrid/search 与 ask 返回 503 |
| 问题在资料中无依据 | 固定文案 + 空引用，不调用对话模型 |
| 向量点残留（删除失败） | MySQL 二次班级复核天然过滤脏 id，不会串班 |

## 四、接口清单（除标注外均需 Bearer Token）

| 接口 | 方法 | 权限 | 说明 / 主要状态码 |
| --- | --- | --- | --- |
| `/auth/login` | POST | 公开 | 账号密码换 token；200 体含 {token,user}；401 凭据错误（响应无 Cookie） |
| `/auth/logout` | POST | 登录 | 删除会话行，token 立即失效；200；401 |
| `/auth/me` | GET | 登录 | 返回当前用户（实时读库）；200；401 |
| `/materials` | GET | 登录 | 本班材料列表 |
| `/materials` | POST | 教师 | 上传并自动建索引；201；403；413；415 |
| `/materials/<id>` | GET | 登录 | 材料详情；跨班/不存在同形 404 |
| `/materials/<id>/download` | GET | 登录 | 下载原文件；跨班同形 404 |
| `/materials/<id>/reindex` | POST | 教师 | 按 auto/custom/hierarchy 重建索引；400 参数错；403；404 |
| `/search` | POST | 登录 | {query, mode=keyword\|vector\|hybrid}；空 query 400；依赖故障 503；无命中 200+空 |
| `/ask` | POST | 登录 | {question, history?}；400；503；无依据 200+固定文案+空引用 |
| `/health` | GET | 公开 | 存活检查（不含 DB 探活） |
| `/login` 、 `/dashboard` | GET | 公开页面 | 页面外壳；未登录由前端 JS 守卫跳登录 |

## 五、验证情况

- **单元/集成测试：**容器内 pytest **76/76 通过**，含 JWT 三段式结构、无 Set-Cookie、payload 仅含 sub/jti/iat/exp、篡改签名与过期 token 被拒、登出即失效、改库后身份实时生效、上传事务、三检索模式、RRF、跨班空命中、无依据不调模型等用例。
- **端到端：**test-client 模式 **29/29**、经 Nginx 的 HTTP 模式 **14/14** 通过。
- **浏览器取证：**登录后 DevTools 可见 `localStorage.cc_token` 为三段式 JWT；Network 中 /auth/me、/materials 等请求均携带 `Authorization: Bearer eyJ…` 且响应 200；登出后该头请求变为 401 并跳回登录页，与老师样例一致。

---

本文全部结论均依据仓库现有代码整理，关键入口： `app/services/token_auth.py` 、 `app/middleware/auth.py` 、 `app/routes/auth.py` 、 `app/repositories/sessions.py` 、

`app/services/{chunking,indexing,retrieval,answer,model_gateway,vector_store}.py` 、
`app/routes/{materials,search,ask}.py` 、 `migrations/001-003*.sql` 。